



南開大學
Nankai University

人工智能技术实验 实验报告

实验名称：基于遗传算法的旅行商问题

姓 名：殷家兴

学 号：2313666

专 业：自动化

人工智能学院

2025 年 12 月

一 问题简述

旅行商问题：设有 N 个互相可直达的城市，给定每对城市之间的距离，某推销商准备从其中的 A 城出发，周游各城市一遍，最后又回到 A 城，要求访问每一座城市并回到起始城市的最短回路。这是非常经典的组合优化问题中的 NP 难问题（多项式复杂程度的非确定性问题），通过常规的暴力枚举，遍历等方法难以计算出最优解，因此，本实验使用遗传算法进行最优解的计算。

遗传算法是根据大自然中生物体进化规律而设计提出的，是模拟达尔文生物进化论的自然选择和遗传学机理的生物进化过程的计算模型，是一种通过模拟自然进化过程搜索最优解的方法。该算法通过数学的方式，利用计算机仿真运算，将问题的求解过程转换成类似生物进化中的染色体基因的交叉、变异等过程。在求解较为复杂的组合优化问题时，相对一些常规的优化算法，通常能够较快地获得较好的优化结果。

对于本实验，求解我国 34 个城市的旅行商问题，给定 34 个城市的名称和坐标数据，求遍历一次所有城市回到起点期间，所经过路程的较小值。

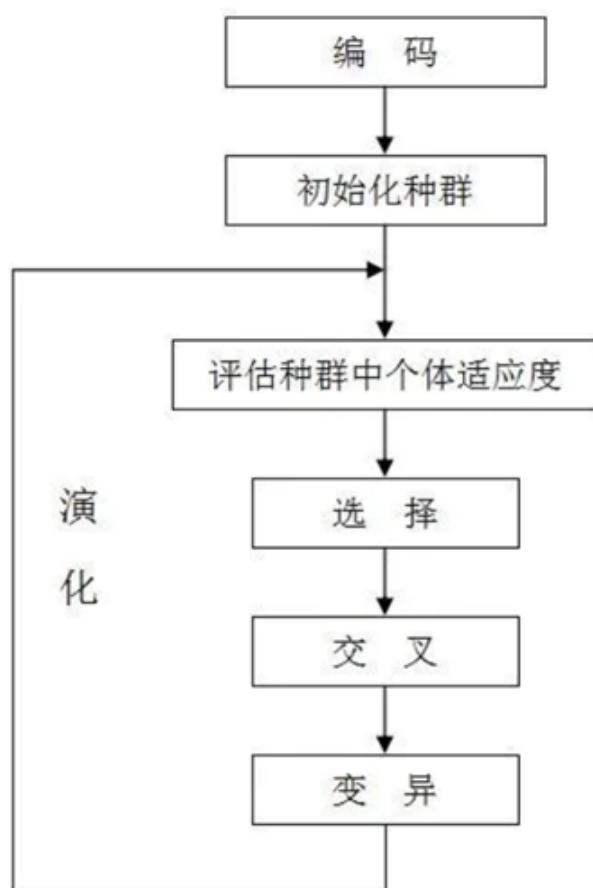


图 1: 算法实现流程

算法实现流程：

1. 初始化阶段：

初始化城市数据：城市数目、城市位置、城市距离矩阵；

初始化基本参数：种群数目、交叉概率、变异概率、交叉方式、变异方式；

初始化种群：随机生成 n 个城市序列， n 为种群数目。

2. 计算种群适应度：根据城市序列和城市距离矩阵计算种群适应度。

3. 生成下一代种群（选择、交叉、变异）

选择：轮盘赌选择、锦标赛选择等方式；

交叉：单点交叉、两点交叉等方式；

变异：均匀变异、基本位变异等方式。

4. 迭代操作：重复步骤 2 和 3，并记录每一次迭代的最优解

5. 结果输出：输出迭代过程中产生的最短路径长度、最短路径；

二 实验目的

1. 了解旅行商问题的基本概念，理解旅行商问题的求解难度；

2. 了解遗传算法的基本原理及实现流程；

3. 能够利用遗传算法解决旅行商问题；

4. 进一步理解可视化设计。（选做）

三 实验内容

实验包括自主起降、路径规划、遍历覆盖、编队协同四个子任务，每个子任务的实验内容如下：

1. 通过基于位置/图像的视觉伺服控制方法实现无人机自主起降功能；

2. 通过基于贝塞尔平滑的多端连续加速度级轨迹规划方法实现多途径点的路径规划；

3. 通过遗传算法实现视觉遍历覆盖；

4. 通过蚁群算法实现多无人机编队协同智能任务规划算法。

四 实验内容

1. 实现基于穷举法的 10 个城市 (No.1-No.10) 的旅行商问题，求解访问每一座城市一次并最终回到起始城市的最短回路，理解穷举法的局限性；
2. 实现基于遗传算法的 34 个城市的旅行商问题，求解访问每一座城市一次并最终回到起始城市的最短回路；
3. 对比遗传算法下不同种群规模、交叉概率、变异概率等参数下，遗传算法对问题求解的效果影响并分析原因。
4. 进行遗传算法可视化，展示搜索过程中每代种群最优路径长度的变化和迭代最优结果。(选做)

五 编译环境

- 操作系统：Windows 11；
- Python 版本：3.9.23；
- 依赖库：numpy、PIL (Pillow)、random、time、os、sys；
- 集成开发环境 (IDE)：VS Code。

六 实验步骤

6.1 使用穷举法求解前十座城市的旅行商问题

起点固定为北京，遍历其余 9 个城市的全排列：

```
1 import itertools
2 import time
3 import numpy as np
4
5 # 城市坐标 (示例：前10个)
6 cities = [
7     [116.4, 39.9], # 北京
8     [121.5, 31.2], # 上海
9     [113.3, 23.1], # 广州
10    [114.3, 30.6], # 武汉
11    [104.1, 30.7], # 成都
12    [106.7, 26.6], # 贵阳
13    [102.7, 25.0], # 昆明
```

```

14     [117.2, 31.8],    # 合肥
15     [118.8, 32.1],    # 南京
16     [126.6, 45.8]     # 哈尔滨
17 ]
18
19 # 构建距离矩阵
20 def calculate_distance_matrix(cities):
21     c = np.array(cities)
22     return np.sqrt(((c[:, None, :] - c[None, :, :]) ** 2).sum(axis=2))
23
24 dist_matrix = calculate_distance_matrix(cities)
25 n = len(cities)
26
27 start_time = time.time()
28 min_cost = float('inf')
29 best_path = None
30
31 # 固定起点为0, 遍历其余城市的全排列
32 for perm in itertools.permutations(range(1, n)):
33     path = (0,) + perm + (0,)
34     cost = sum(dist_matrix[path[i]][path[i+1]] for i in range(n))
35     if cost < min_cost:
36         min_cost = cost
37         best_path = path
38
39 end_time = time.time()
40
41 print(f"最小开销: {min_cost:.2f}")
42 print(f"最优路径: {best_path}")
43 print(f"运行时间: {end_time - start_time:.2f} 秒")

```

6.2 使用遗传算法求解三十四座城市的旅行商问题

6.2.1 城市数据加载与距离矩阵构建

程序内置中国 34 个主要城市的经纬度坐标, 并通过计算两两点位之间的欧氏距离矩阵:

```

1 def _calculate_distance_matrix(self, cities):
2     cities_array = np.array(cities, dtype=np.float32)
3     diff = cities_array[:, np.newaxis, :] - cities_array[np.newaxis, :, :]
4     return np.sqrt(np.sum(diff ** 2, axis=2))

```

6.2.2 种群初始化

每个个体表示一条合法路径，固定起点和终点为北京（索引 0），同时确保每条路径访问所有城市恰好一次，且构成回路。

```
1 path = list(range(1, self.n_cities))
2 random.shuffle(path)
3 individual = [0] + path + [0]
```

6.2.3 遗传操作实现

- **选择**：采用轮盘赌选择，并强制保留每代前 25% 的精英个体，防止优质解丢失。

```
1 def selection(self, ranked_pop):
2     elites = [ranked_pop[i][0] for i in range(self.elite_size)]
3     # 轮盘赌选择剩余个体
4     df = pd.DataFrame(ranked_pop, columns=["Index", "Fitness", "
Distance"])
5     total_fitness = df["Fitness"].sum()
6     prob = df["Fitness"] / total_fitness
7     selected_indices = np.random.choice(df["Index"], size=self.
pop_size - self.elite_size, p=prob)
8     return elites + selected_indices.tolist()
9
```

- **交叉**：使用有序交叉（Order Crossover, OX）。随机选取一段子序列复制到子代，其余位置按父代 2 的顺序填充未出现的城市，保证无重复。

```
1 def crossover(self, parent1, parent2):
2     start, end = sorted(random.sample(range(1, len(parent1)-1), 2)
3 )
4     child = [None] * len(parent1)
5     child[0] = child[-1] = 0
6     child[start:end] = parent1[start:end]
7
8     pointer = end
9     for city in parent2[end:-1] + parent2[1:end]:
10         if city not in child:
11             child[pointer] = city
12             pointer = pointer + 1 if pointer + 1 < len(child) - 1
13         else:
14             continue
15     return child
```

- **变异**：以设定概率执行以下任一操作：

交换变异：随机交换路径中两个非起点/终点城市；

逆序变异：随机选取一段子路径并反转，增强局部搜索能力。

```
1     def mutate(self, individual, mutation_rate=0.03):
2         if random.random() < mutation_rate:
3             # 50% 概率执行交换变异，50% 执行逆序变异
4             if random.random() < 0.5:
5                 i, j = random.sample(range(1, len(individual)-1), 2)
6                 individual[i], individual[j] = individual[j],
individual[i]
7             else:
8                 i, j = sorted(random.sample(range(1, len(individual)
-1), 2))
9                 individual[i:j] = reversed(individual[i:j])
10            return individual
11
```

6.2.4 迭代进化与记录

主循环运行指定代数，每 100 代打印当前最优路径长度与平均长度，便于观察收敛趋势，最终返回历史最优个体。

主循环如下：

```
1 def evolve(self, generations=1000, cxpb=0.65, mutpb=0.03):
2     pop = self.create_population()
3     best_distance = float('inf')
4     best_route = None
5
6     for gen in range(generations):
7         ranked = self.rank_population(pop)
8         current_best = ranked[0][2]
9         if current_best < best_distance:
10             best_distance = current_best
11             best_route = pop[ranked[0][0]]
12
13         selected = self.selection(ranked)
14         children = []
15         for i in range(self.elite_size):
16             children.append(pop[selected[i]]) # 精英直接保留
17         for i in range(self.elite_size, self.pop_size):
18             parent1 = pop[random.choice(selected)]
19             parent2 = pop[random.choice(selected)]
20             if random.random() < cxpb:
21                 child = self.crossover(parent1, parent2)
22             else:
```

```

23         child = parent1[:]
24         child = self.mutate(child, mutpb)
25         children.append(child)
26     pop = children
27
28     if gen % 100 == 0:
29         avg_dist = np.mean([r[2] for r in ranked])
30         print(f"代 {gen}: 最优={current_best:.2f}, 平均={avg_dist:.2f}"
31       )
32
33     return best_route, best_distance

```

修改种群规模、迭代次数、交叉概率和变异概率四种参数，总共得到七种解决方案。

6.2.5 结果可视化

记录通过上述过程所求得的最小代价方案，并将其写入“可视化_TSP.py”中，绘制路线图。

七 实验结果

7.1 穷举法求解前十座城市旅行商问题结果

最终得到的路线为：

北京 → 天津 → 哈尔滨 → 上海 → 南宁 → 重庆 → 拉萨 → 乌鲁木齐 → 银川 → 呼和浩特 → 北京

最小开销为 110.35。

穷举法在城市数量相同的情况下，计算量相较于遗传算法更大，计算时间更长，如果增加到 34 个城市使用穷举法，使用的时间与计算量将成呈爆炸式增长。

7.2 使用遗传算法求解三十四座城市的旅行商问题

表 1: 遗传算法求解旅行商问题结果

	1	2	3	4	5	6	7
种群规模	80	80	80	100	100	100	100
迭代次数	100	100	500	500	1000	1000	1000
交叉概率	0.60	0.60	0.60	0.60	0.65	0.65	0.70
突变概率	0.020	0.030	0.030	0.030	0.035	0.030	0.040
运行时间	0.07	0.06	0.33	0.42	0.85	0.83	0.86
最小开销	238.85	236.88	207.07	175.97	180.77	170.75	178.67

其中，最小开销最小的情况为第六次运行结果，本次方案如图2所示。

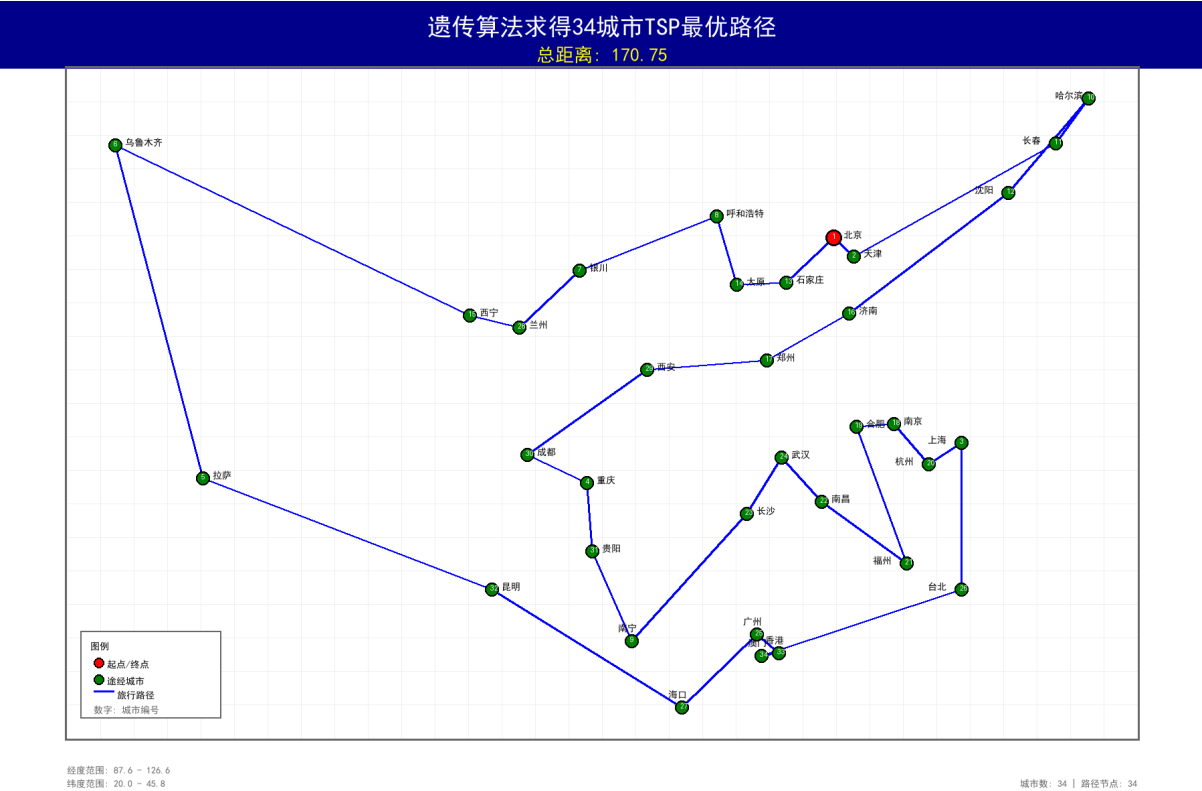


图 2: 第六次规划方案

基于表1中结果，分析各参数对于结果的影响：

1. 种群规模的影响

对比第 1、2、3 组与第 4、5、6、7 组可知，种群规模从 80 增至 100 后，整体最优解（最小开销）明显改善（如第 3 组 207.07 对比第 4 组 175.97）。种群规模增

大，样本多样性提升，有助于跳出局部最优，但计算时间相应增加（如从 0.33 秒增至 0.42 秒）。

2. 迭代次数的影响

在相同种群规模下，迭代次数增加能显著提升解的质量。例如第 1 组（100 代）与第 3 组（500 代）对比，最优解从 238.85 降至 207.07；第 4 组（500 代）与第 6 组（1000 代）对比，最优解从 175.97 降至 170.75。更多迭代为算法提供了更充分的进化空间，有利于收敛至更优解。

3. 交叉概率的影响

对比第 5 组（0.65）与第 6 组（0.65）以及第 7 组（0.70），交叉概率在一定范围内提升有助于种群信息的交换与重组，但过高可能破坏优良基因结构。第 6 组在适中交叉概率（0.65）下取得了最优结果（170.75），说明合适的交叉概率对平衡探索与利用至关重要。

4. 变异概率的影响

变异概率较低时（如 0.02），种群多样性不足，易早熟收敛（如第 1 组 238.85）；适度提高（如 0.03 0.04）可增强局部搜索能力，帮助跳出局部最优（如第 6 组 170.75）。但过高变异概率可能导致优良基因丢失，增加随机性，影响收敛稳定性。

八 分析总结

通过对比遗传算法求解旅行商问题得到的 7 组不同参数下的运行结果，可以看出，种群规模、迭代次数、交叉概率和变异概率对算法性能具有显著影响。种群规模从 80 增至 100 后，解的质量明显提升（如第 4 组 175.97 优于第 3 组 207.07），但计算时间略有增加；迭代次数增加同样能优化结果，如第 6 组（1000 代）得到 170.75 的最优解，明显优于第 4 组（500 代）的 175.97。交叉概率在适中范围（0.65 左右）时效果最好，过高可能破坏优良基因结构；变异概率适度提高（0.03 0.04）有助于增强局部搜索能力、避免早熟收敛，但过高会导致随机性增加、收敛不稳定。

与穷举法相比，遗传算法在大规模问题上优势明显：穷举法虽然能保证 10 个城市时的精确最优解，但其计算量随城市数增长呈阶乘级爆炸，无法应用于 34 个城市的实际求解；而遗传算法在很短时间内（如第 6 组 0.83 秒）就能获得满意解，体现了遗传算法的实用性和高效性。